

Lecture 02

Syntax, Semantics, Environments:
Expressions, Functions

The Recipe: Syntax + Semantics

- Syntax: how programs are written down
 - Roughly “spelling and grammar”
- Semantics: what programs *mean*
 - Type checking : compile time (aka “static”) semantics
 - Evaluation : run time (aka “dynamic”) semantics
- Both syntax and semantics matter
 - 341’s view $\approx$ “Semantics give the essence of a PL.”
 - 341’s view $\approx$ “Syntax is more about taste.”

Why Two Semantics?

- Type checking (static semantics) first makes a guarantee
 - Proves invariant!
 - Happens before a program starts to run
- Evaluation (dynamic semantics) then runs the program
 - Because program type checks, many run-time errors **impossible**
 - Example: `1 + "hello"`
 - There is no dynamic semantics for this [non-]program
 - Will never run a program that evaluates such an expression

OCaml Carefully So Far

- *A program is just a sequence of bindings*
- Typecheck each binding in order
 - Using **static environment** from previous bindings
- Evaluate each binding in order
 - Using **dynamic environment** from previous bindings
- So far, only variable bindings
 - More kinds of bindings coming soon!

Expressions

- Many kinds of expressions in OCaml
 - 34 true xyz
 - $e1 + e2$ $e1 < e2$ if $e1$ then $e2$ else $e3$
- *Recursively* defined → can be arbitrarily large
 - Expressions can have *subexpressions* that have *subsubexpressions* that have ...

Defining Expressions: Syntax, Typing, Evaluation

- Syntax : how to write down that kind of expression (keywords, etc.)
- Semantics : what that kind of expression *means*
 - Type checking rules (“static semantics”)
 - Produces a type or reports an error
 - Evaluation rules (“dynamic semantics”)
 - Produces a value (or runs forever or raises an exception)
 - *Only defined for well-typed expressions!*

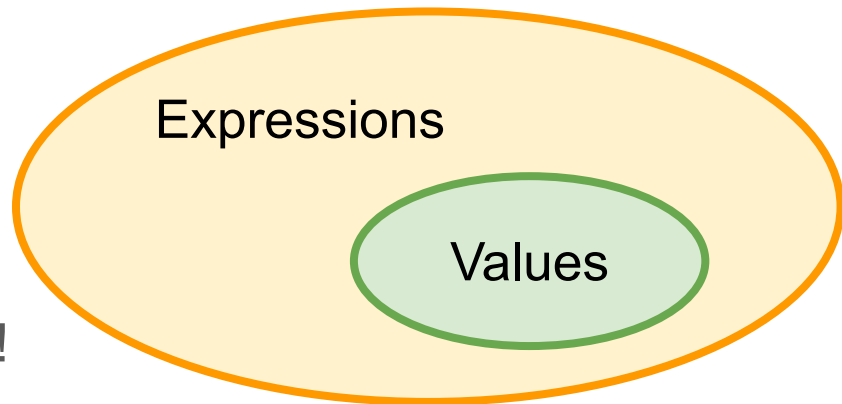
Defining Expressions: Syntax, Typing, Evaluation

- Syntax : how to write down that kind of expression (keywords, etc.)
- Semantics : what that kind of expression means
 - Type checking rules (“static semantics”)
 - Produces a type or reports an error
 - Evaluation rules (“dynamic semantics”)
 - Produces a value (or runs forever or raises an exception)
 - *Only defined for well-typed expressions!*

Systematic approach to
defining / understanding
programming languages!

Values

- All values are expressions
 - But not all expressions are values!
- Values are the “answers” returned by evaluation
 - A value always evaluates to itself immediately
- Examples
 - `12 27 13` are values of type `int`
 - `true false` are (the only) values of type `bool`



Variables

Syntax: sequence of letters, numbers, `_`, or `'`

- Must start with a letter or `_`

Type Checking

- Look up in current static environment
- If found, return corresponding type
- Else fail

Evaluation

- Look up in current dynamic environment
- Return corresponding value

Variables

Syntax: sequence of letters, numbers, `_`, or `'`

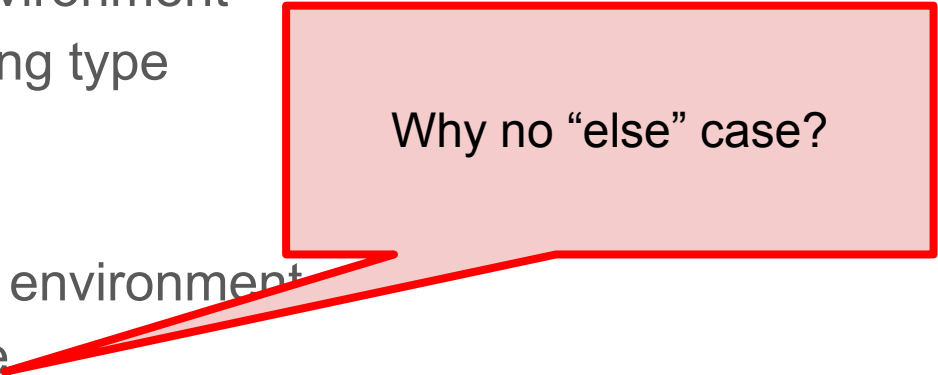
- Must start with a letter or `_`

Type Checking

- Look up in current static environment
- If found, return corresponding type
- Else fail

Evaluation

- Look up in current dynamic environment
- Return corresponding value



Why no “else” case?

Variables

Syntax: sequence of letters, numbers, `_`, or `'`

- Must start with a letter or `_`

Type Checking

- Look up in current static environment
- If found, return corresponding type
- Else fail

Evaluation

- Look up in current dynamic environment
- Return corresponding value

Why no “else” case?

Type checking ensures
variable always bound!

Addition

Syntax: $e1 + e2$

- Where $e1$ and $e2$ are expressions

Addition

Syntax: $e1 + e2$

- Where $e1$ and $e2$ are expressions

Type Checking

- If $e1$ has type int and $e2$ has type int
- Then $e1 + e2$ has type int
- Else, report error and fail

Addition

Syntax: $e1 + e2$

- Where $e1$ and $e2$ are expressions

Type Checking

- If $e1$ has type int and $e2$ has type int
- Then $e1 + e2$ has type int
- Else, report error and fail

Evaluation

- If $e1$ evaluates to value $v1$ and $e2$ evaluates to value $v2$
- Then $e1 + e2$ evaluates to the sum of $v1$ and $v2$

Addition

Syntax: $e1 + e2$

- Where $e1$ and $e2$ are expressions

Type Checking

- If $e1$ has type int and $e2$ has type int
- Then $e1 + e2$ has type int
- Else, report error and fail

But what if $e1$ or $e2$ do not evaluate to $ints$?

Evaluation

- If $e1$ evaluates to value $v1$ and $e2$ evaluates to value $v2$
- Then $e1 + e2$ evaluates to the sum of $v1$ and $v2$

Addition

Syntax: $e1 + e2$

- Where $e1$ and $e2$ are expressions

Type Checking

- If $e1$ has type int and $e2$ has type int
- Then $e1 + e2$ has type int
- Else, report error and fail

But what if $e1$ or $e2$ do not evaluate to $ints$?

Evaluation

- If $e1$ evaluates to value $v1$ and $e2$ evaluates to value $v2$
- Then $e1 + e2$ evaluates to the sum of $v1$ and $v2$

Type checking ensures they will be $ints$!

Group Work

Syntax: $e1 < e2$

Type Checking

Evaluation

Syntax: if $e1$ then $e2$ else $e3$

Type Checking

Evaluation

Addition

Syntax: $e1 + e2$

- Where $e1$ and $e2$ are expressions

Type Checking

- If $e1$ has type int and $e2$ has type int
- Then $e1 + e2$ has type int
- Else, report error and fail

Evaluation

- If $e1$ evaluates to value $v1$ and $e2$ evaluates to value $v2$
- Then $e1 + e2$ evaluates to the sum of $v1$ and $v2$

Comparison (less than)

Syntax: $e1 < e2$

- Where $e1$ and $e2$ are expressions

Type Checking

- If $e1$ has type int and $e2$ has type int
- Then $e1 < e2$ has type $bool$
- Else, report error and fail

Evaluation

- If $e1$ evaluates to value $v1$ and $e2$ evaluates to value $v2$
- Then $e1 < e2$ evaluates to **true** if $v1$ is less than $v2$, and **false** otherwise

Conditionals (if-then-else)

Syntax: `if e1 then e2 else e3`

- Where `e1`, `e2`, and `e3` are expressions

Type Checking

- If `e1` has type `bool` and `e2` and `e3` have the same type `t`
- Then `if e1 then e2 else e3` has type `t`
- Else, report error and fail

Evaluation

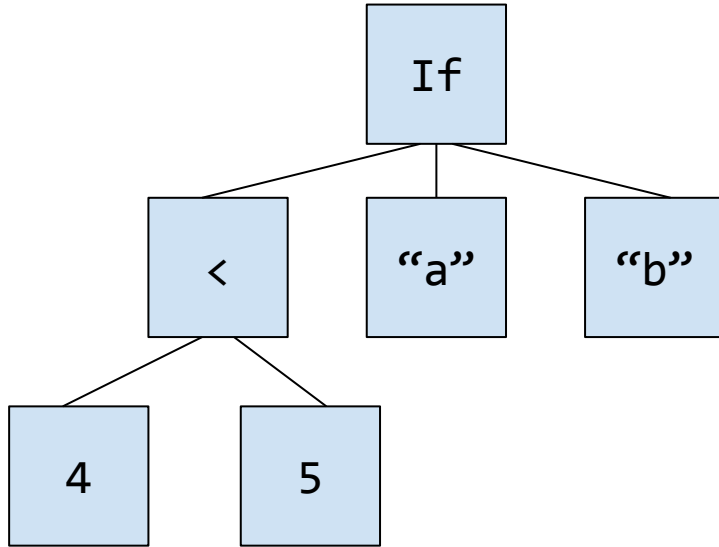
- If `e1` evaluates to **true**, then return result of evaluating `e2`
- If `e1` evaluates to **false**, then return result of evaluating `e3`

Both branches must have the same type!

Only evaluate one of the branches!

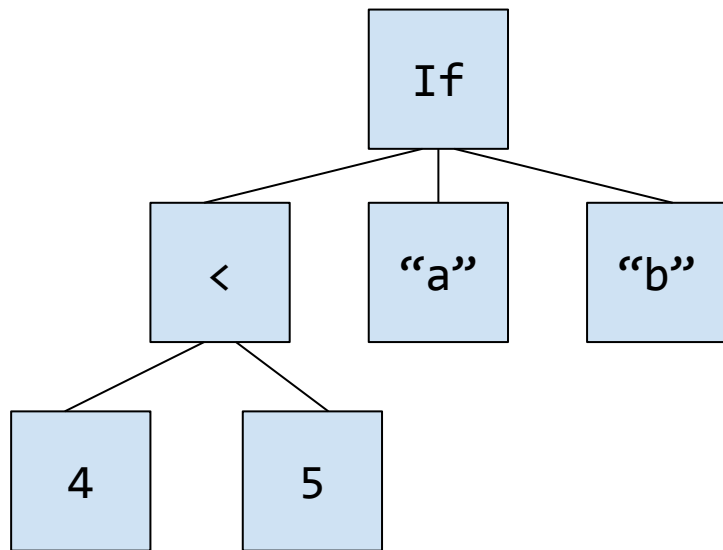
Programs are Trees

if (4 < 5) then "a" else "b"



Programs are Trees

if (4 < 5) then "a" else "b"



Group Work

Draw the following programs as trees:

$(1 + 2) * (3 * 4)$

$1 + ((2 * 3) + 6)$

if true then
 if false then 3 else 4
else 5

The Foundation We Need

- *Lots* more types, expressions, and bindings
 - “Interesting” programs soon 😊
- Syntax, type checking, and evaluation will continue to guide us
 - For HW1: functions, tuples, lists, options, local bindings
 - Earlier problems need fewer features
- Will **not** need
 - **Mutation** (aka assignment): use **recursion** and/or **new variables**
 - **Statements**: use **expressions**
 - **Loops**: use **recursion**

Functions

- Most important building block in the whole course
- Abstract computation by parameterizing over parts of an expression
- Like Java methods, have arguments and result
 - But no classes, *this*, *returns*, etc.
- Example *function binding* and *function call*

```
let max ((x:int),(y:int)) =  
    if x > y then x else y
```

```
max (3,17)
```

Example With Recursion

```
let rec pow ((x:int),(y:int)) =  
    if y = 0  
    then 1  
    else x * pow (x, y-1)  
let cube (x:int) =  
    pow (x, 3)  
let sixtyfour = cube 4  
let fortytwo = pow (2, 4) + pow (4, 2) + cube 2 + 2
```


Heads Up: Recursion!

- If you're not comfortable with recursion yet, you will be soon 😊
 - We'll write *many* recursive functions in lecture and homework
 - Mostly over recursive data structures like lists (next lecture)
- “Makes sense” because recursive calls solve “simpler” problems
- Recursion “more general” than loops
 - We won't write a single loop in OCaml
 - Easy to simulate loops with recursion
 - Loops often (not always) obscure simple, elegant solutions

Six questions

Three questions for a new language construct:

syntax

typing rules

evaluation rules

We have two new language constructs:

function bindings

function calls

Function Bindings

Syntax, with or without rec:

let f $((x_1 : t_1), \dots, (x_N : t_N)) = e$

let rec f $((x_1 : t_1), \dots, (x_N : t_N)) = e$

Where:

- $f, x_1, \dots, x_N$ are variable names
- $t_1, \dots, t_N$ are types
- e is an expression

Meta-syntax

- We use [...] to indicate some syntax is optional
 - Like `rec`
 - So the syntax for function bindings is

`let [rec] f ((x1 : t1), ..., (xN : tN)) = e`

- The presence/absence of `rec` *does* matter
 - It affects the typing rules and evaluation rules!
 - Just saying it is correct function-binding *syntax* with or without it

Actually, part 1

This is a simplified-for-now syntax story:

1. Can write the return type if you want (example code won't):
`let [rec] f ((x1 : t1), ..., (xN : tN)) : t = e`
2. Can omit argument types (and we will *after* Homework 1):
`let [rec] f (x1, ..., xN) = e`
3. Will see a different, more common, way to do multiple arguments later

Note: Be careful on argument syntax!

Slight errors can produce strange messages or behavior

Actually, part 2

For one-argument functions, can have fewer parentheses (don't need to):

```
let [rec] f ((x1 : t1)) = e
```

```
let [rec] f (x1 : t1) = e
```

```
let [rec] f (x1) = e
```

```
let [rec] f x1 = e
```

But NOT `let [rec] f x1 : t1 = e` (* means *returns a t1* *)

Function Bindings: Type Checking

$\text{let } [\text{rec}] \ f \ ((x_1 : t_1), \dots, (x_N : t_N)) = e$

Type-check e in the static environment containing:

- The “enclosing” (up to this point) static environment
- $x_1 \mapsto t_1, \dots, x_N \mapsto t_N$ arguments and their types
- If “rec” is given, $f \mapsto t_1 * \dots * t_N \rightarrow t$

If e has type t in this static environment

Then add $f \mapsto t_1 * \dots * t_N \rightarrow t$ to the outer static environment

Else, report error and fail

Function Bindings : Need Function Types!

- New kind of type! $f : t_1 * \dots * t_N \rightarrow t$
- Argument types $t_1, \dots, t_N$ separated by $*$
- Result type t after the arrow

How this is used for type-checking function bindings:

- f has this type in the rest of program (not in earlier bindings!)
- Arguments available in e , but not after (unsurprising)
- Calling f evaluates e , so return type of f is type of e
- “Magic” for now (study in a few weeks!):

Type-checker figures out t even when f is recursive

Function Bindings: Evaluation Rules

`let [rec] f ((x1 : t1), ..., (xN : tN)) = e`

Evaluation rules:

- *Nothing to do! Function are values!*
 - Real story a bit more nuanced, but we will get to it 😊
- Add `f` to the dynamic environment so later expressions can call
 - `f` is bound to the function being defined here
- And for recursion, if `rec` is present, then `f` also bound within `e`

Function Calls

Syntax: $f(e_1, \dots, e_N)$

Where $f, e_1, \dots, e_N$ are expressions

- (will generalize this a bit later)

Type Checking:

If:

- f has type $f : t_1 * \dots * t_N \rightarrow t$
- e_1 has type t_1
- $\dots$
- e_N has type t_N

Then $f(e_1, \dots, e_N)$ has type t

Function Calls

$e_0 (e_1, \dots, e_N)$

Evaluation

1. Evaluate e_0 in current dynamic environment
 - Since e_0 type checked, the result will be a function
 - The result is some $\text{let } [\text{rec}] \ f \ ((x_1:t_1), \dots, (x_N:t_N)) = e$
2. In current dynamic environment, evaluate e_1 to value v_1 , ... , e_N to value v_N
3. The $e_0 (e_1, \dots, e_N)$ call now evaluates to the result of evaluating e
In an extended dynamic environment with $x_1 \mapsto v_1$, ... , $x_N \mapsto v_N$

Function Calls

$e_0 (e_1, \dots, e_N)$

Evaluation

1. Evaluate e_0 in current dynamic environment
 - Since e_0 type checks, we know it is a function
 - The result is some `let [rec] f ((x1:t1), ..., (xN:tN)) = e`
2. In current dynamic environment, evaluate e_1 to value v_1 , ..., e_N to value v_N
3. The $e_0 (e_1, \dots, e_N)$ call now evaluates to the result of evaluating e *In an extended dynamic environment with $x_1 \mapsto v_1$, ..., $x_N \mapsto v_N$*

Technically:

extend dynamic environment where f was defined.

But more on this later!

Heads Up: Function Gotchas

- Be careful with argument declaration and argument passing
 - Else confusing error messages
- Remember: cannot refer to later bindings
 - Helper functions must come first
 - Mutual recursion is handled specially

So much power in 6 questions

3 questions per language construct:

- *syntax*
- *typing rules*
- *evaluation rules*

2 new language constructs:

- *function bindings*
- *function calls*

Most important building block of 341!

```
let rec pow ((x:int),(y:int)) =  
    if y = 0  
    then 1  
    else x * pow (x,y-1)  
let cube (x:int) =  
    pow (x,3)  
let sixtyfour = cube 4
```